



400-650-7088

***Tong*RDS** V2.2

监控管理手册

版本：2218A01

2025 年 9 月

声明

版权

Copyright ©2025 北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

商标声明

- **东方通**、**TongTech**标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

本手册版本（2218A01）适用于 TongRDS V2.2.1.8。

文档版本	适用产品版本	更新内容	更新日期
2218A01	V2.2.1.8	更新如下内容： 更新版权声明。	2025/9/29
2217P1A01	V2.2.1.7_P1	新增如下内容： ● 3.2 数据暴露方式：新增metrics2cn接口的全节点数据获取方式； ● 3.3 数据暴露方式的区分：新增章节； ● 3.4.2 metrics2cn接口prometheus.yaml配置：新增章节。	2025/6/13
2217A01	V2.2.1.7	无	2025/4/18
2216P2A01	V2.2.1.6_P2	无	2024/12/6
2216P1A01	V2.2.1.6_P1	无	2024/9/29
2216A01	V2.2.1.6	无	2024/8/8
2215A01	V2.2.1.5	更新如下内容： 2.5 启停Console-exporter：更新启动时的界面图片。	2024/6/28
2214A01	V2.2.1.4	TongRDS V2.2第一次正式发布。	2023/12/29

前言

本文档是 TongRDS V2.2 产品的用户使用手册之一，提供安装部署监控管理组件的基本步骤。

阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

● 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

● 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

● 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

1. 掌握 Linux 系统基本操作；
2. 熟悉 Java 开发语言。

● 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

● 实践经验

为了最大程度地受益于本文档，建议您具备一定的实践经验。这将帮助您更好地应用文档中的操作指南和建议。

请注意，本文档并不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容或所需背景有任何疑问，请在使用之前咨询相关专业人士或寻求额外的支持。

用户使用手册集

TongRDS V2.2 为您提供的用户使用手册集包含的文档有：

编号	手册集文档	说明
1	000_TongRDS_V2.2 用户手册导读	提供手册集的目录及手册版本说明。
2	001_TongRDS_V2.2 快速使用手册	提供产品介绍，安装部署产品的基本步骤及验证方式。
3	002_TongRDS_V2.2 配置参数手册	提供配置文件及参数的详细说明。
4	003_TongRDS_V2.2 管理控制台安装手册	详细介绍管理控制台的安装卸载、启动停止及登录。

编号	手册集文档	说明
5	004_TongRDS_V2.2 管理控制台使用手册	提供管理控制台基础信息、RDS 管理、系统监控、系统审计等功能的使用说明。
6	005_TongRDS_V2.2 客户端使用手册	介绍 TongRDS 提供的客户端程序使用说明。
7	006_TongRDS_V2.2 监控管理手册	提供安装部署监控管理组件的基本步骤。
8	007_TongRDS_V2.2 场景配置参考手册	提供常见部署模式的配置示例及验证方法。
9	008_TongRDS_V2.2 命令&接口使用手册	提供命令及接口的详细说明及使用方法。
10	009_TongRDS_V2.2 开发手册	详细介绍 TongRDS 提供的开发功能。
11	010_TongRDS_V2.2 运维手册	提供 TongRDS 常见问题的解决方案。

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

网址：www.tongtech.com

Support Tel：400-650-7088

邮箱：support@tongtech.com

您在取得技术支持时，请提供如下信息：

1. 您的姓名
2. 您的公司信息
3. 您的联系方式
4. 硬件及软件信息
5. 操作系统及其版本
6. 产品版本号
7. 日志等错误的详细信息

目录

版本变更说明	3
前言	4
目录	6
第 1 章 概述	7
第 2 章 部署监控管理组件	8
2.1 部署前准备	8
2.1.1 运行环境	8
2.1.2 获取安装包	8
2.2 安装步骤	8
2.3 目录说明	8
2.4 修改配置文件	9
2.5 启停 Console-exporter	9
第 3 章 使用监控管理组件	11
3.1 数据采集指标	11
3.2 数据暴露方式	12
3.3 数据暴露方式的区别	12
3.4 数据采集	13
3.4.1 metrics 接口 prometheus.yaml 配置	13
3.4.2 metrics2cn 接口 prometheus.yaml 配置	14
3.5 数据展示	15
附录 A 术语说明	19

第1章 概述

TongRDS 提供监控管理组件 Console-exporter。Console-exporter 是类似于 Redis-exporter 的 RDS 运行监控程序，其作用为监控 RDS 运行时所记录的一些监控项。Console-exporter 通过访问 RDS 中心节点 restful 接口获取监控数据，然后对每个服务节点的监控数据进行注册。注册在程序中的数据，可通过访问 console-exporter 的 API 接口将数据取出。

说明：监控管理组件非必须安装，请用户根据实际需求选择安装。

第2章 部署监控管理组件

2.1 部署前准备

在虚拟机上部署 TongRDS 监控管理组件前，请先按照《001_TongRDS_V2.2 快速使用手册》部署 TongRDS 并验证成功。

2.1.1 运行环境

TongRDS 的监控管理组件需运行在 64 位 Linux 系统环境下。系统要求满足以下条件：

系统组件	系统要求
硬件	Linux 操作系统所支持的硬件平台。
操作系统	Linux
JAVA 环境	JRE/JDK1.8 及以上版本，建议使用 JRE/JDK 1.8

2.1.2 获取安装包

部署监控管理组件需要获取的安装包如下：

安装包名称	说明	解压后目录名
console-exporter.tar.gz	监控管理组件安装包。	console-exporter

2.2 安装步骤

1. 获取监控管理组件安装包，将安装包上传到服务器的新建用户或已存在的用户目录下，例如：
`/home/xxx`。
2. 进入 `/home/xxx` 目录，执行解压命令：

```
tar -zxvf console-exporter.tar.gz
```

2.3 目录说明

解压监控管理组件安装包后，`/home/xxx/console-exporter` 下的子目录及文件如下。

目录名称	说明
bin	操作脚本（启动、停止、重启）
config	配置文件
logs	日志文件
console-exporter.jar	运行程序

2.4 修改配置文件

1. 进入配置文件目录 console-exporter/config

cd console-exporter/config

2. 在配置文件 **exporter.config** 中修改如下配置:

vi exporter.config

```
exporter.name=${EXPORTER_NAME\:RDS_exporter}

exporter.log_level=${EXPORTER_LOG_LEVEL\:info}

exporter.service_port=${EXPORTER_SERVICE_PORT\:9091}

exporter.center_host=${CENTER_ADMIN_URL\:127.0.0.1,192.168.0.86}

exporter.center_admin_port=${CENTER_ADMIN_PORT\:9744,8086}

exporter.center_auth_key=${CENTER_AUTH_KEY\:abcef12376se790fowieaawew90qa8ew8e}
```

参数说明如下:

- **exporter.name:** console-exporter 的名称。
- **exporter.log_level:** 程序的日志级别。
- **exporter.service_port:** 程序 web 服务的端口。
- **exporter.center_host:** 监控的中心节点地址 (多个 ip 使用 “,” 分隔)。
- **exporter.center_admin_port:** 监控的中心节点端口 (和上述 ip 对应的端口号)。
- **exporter.center_auth_key:** 监控的中心节点授权 Token。

2.5 启停Console-exporter

- **启动 console-exporter:**

在服务器的 console-exporter/bin 目录下, 运行以下命令启动 Console-exporter:

./starter.sh

```
[root@app-test2 bin]# sh starter.sh
Start console-exporter.jar success...
[root@app-test2 bin]#
```

查看日志: 在 **console-exporter/logs** 目录下生成 **exporter.log.xxxxxxxx** 日志文件。

- **停止 console-exporter:**

在服务器的 console-exporter/bin 目录下, 运行以下命令停止 Console-exporter:

./stop.sh

```
[root@node2 bin]# ./stop.sh
console-exporter.jar (pid:47547) exiting...
console-exporter.jar exited.
[root@node2 bin]#
```

第3章 使用监控管理组件

3.1 数据采集指标

Console-exporter 会采集监控的中心节点所管理的所有服务节点信息，其中不同节点的相同监控项以 Label 和后缀名进行区分，每项监控项设置 target (pod, instance, namespace, S13_test12)，分别代表节点名称、实例名称、空间名称，节点的空间名称，以 namespace 为后缀，例：db_keys 监控项，具体为 redis_db_keys_S13_test13{pod="S13_test13-192.168.0.86:6323",instance="rds://192.168.0.86:6323",namespace="S13_test13",}。

采集注册的数据项有：

监控项	名称	说明
redis_db_keys	Key 数量	监控的是服务节点所有 key 的数量
redis_connected_clients	客户端连接数	当前连接服务节点的客户端数量
redis_connections_received_total	客户端连接数总量	接收到的客户端连接数总量
redis_memory_used_bytes	使用内存	服务节点当前存储数据使用的内存大小
redis_cpu_sys_seconds_total	系统 CPU 使用量	-
redis_cpu_user_seconds_total	用户 CPU 使用量	-
redis_memory_used_rss_bytes	服务节点使用的内存	包含数据使用和程序运行使用
redis_memory_max_bytes	最大内存	设置的 JVM 最大内存值
redis_memory_system_total	系统内存	服务器内存大小
redis_memory_free_bytes	空闲内存	JVM 空闲内存大小
redis_slow_logs	慢操作数	超过设置的慢操作时间的数量
redis_license_occupied	授权使用量	服务节点使用授权的大小
redis_memcache_connections	缓存连接数	-
redis_connections	RESP 协议客户端连接数	仿 redis 端口连接的客户端数量
redis_rds_connections	RDSCP 协议客户端连接数	RDS 自有协议（6200）连接的数量
redis_input_per_second	输入速率	-
redis_output_per_second	输出速率	-
redis_net_input_bytes_total	网络输入大小	-
redis_net_output_bytes_total	网络输出大小	-
redis_commands_processed_total	接收命令总数	-
redis_uptime_in_seconds	运行时长	-
redis_commands_total	执行的总命令数	-
redis_expired_keys_total	过期 key 数量	-
redis_evicted_keys_total	被驱逐 key 数量	-
redis_db_keys_expiring	设置过期 key 数量	-

监控项	名称	说明
redis_keyspace_hits_total	命中数	-
redis_keyspace_misses_total	未命中数	-

3.2 数据暴露方式

数据是通过 web 接口暴露外部访问的，可以通过 **metrics** 接口和 **metrics2cn** 接口两种方式进行数据获取。

● 方式一：metrics 接口

○ 全节点数据获取

请求路径：http://localhost:port/metrics

返回的数据格式为：

```
redis_db_keys_S13_test13{pod="S13_test13-192.168.0.86:6323",instance="rds://192.168.0.86:6323",namespace="S13_test13",}
```

○ 具体节点数据获取

请求路径：http://localhost:port/metrics?target=rds://127.0.0.1:6379

返回的数据格式为：

```
redis_db_keys{pod="S13_test13-192.168.0.86:6323",instance="rds://192.168.0.86:6323",namespace="S13_test13",}
```

● 方式二：metrics2cn 接口

全节点数据获取：

请求路径：http://localhost:port/metrics2cn

返回的数据格式为：

```
redis_db_keys{pod="S13_test13-192.168.0.86:6323",instance="rds://192.168.0.86:6323",namespace="S13_test13",}
```

3.3 数据暴露方式的区别

Exporter 提供两种数据暴露方式，即 **metrics** 接口和 **metrics2cn** 接口。以下说明两种数据暴露方式的不同点：

1. 单条数据返回格式不同。以 redis_db_keys 为例：

● metrics 返回格式如下：

```
redis_db_keys_testtt_zzzddd{pod="testtt_zzzddd-192.168.0.200:6370",instance="testtt-zzzddd",namespace="testtt_zzzddd",service="testtt",} 0.0
```

● metrics2cn 返回格式如下：

```
redis_db_keys{pod="testtt_zzzdddd-192.168.0.200:6370",instance="testtt-zzzdddd",namespace="testtt_zzzdddd",service="testtt",} 0.0
```

主要区别在于：指标项 **redis_db_keys** 在 **metrics** 接口下会携带 **instance** 后缀（**redis_db_keys_testtt_zzzdddd**）；在 **metrics2cn** 的接口下不会携带 **instance** 后缀。

2. 多个实例返回格式不同：

例如：有 testtt-zzzdddd 和 jkjk-qqqq 两个实例。

- metrics 返回格式如下：

```
redis_db_keys_testtt_zzzdddd{pod="testtt_zzzdddd-192.168.0.200:6370",instance="testtt-zzzdddd",namespace="testtt_zzzdddd",service="testtt",} 0.0
```

```
redis_db_keys_jkjk_qqqq{pod="jkjk_qqqq-192.168.0.200:5379",instance="jkjk-qqqq",namespace="jkjk_qqqq",service="jkjk",} 25.0
```

两个实例分别对应两个指标项目。

- metrics2cn 返回格式如下：

```
redis_db_keys{pod="jkjk_qqqq-192.168.0.200:5379",instance="jkjk-qqqq",namespace="jkjk_qqqq",service="jkjk",} 25.0
```

```
redis_db_keys{pod="testtt_zzzdddd-192.168.0.200:6370",instance="testtt-zzzdddd",namespace="testtt_zzzdddd",service="testtt",} 0.0
```

两个实例对应一个指标项，没有 instance 后缀。

3.4 数据采集

Console-exporter 的监控数据可以传输到 prometheus，这里以配置 prometheus 获取数据为例。

3.4.1 metrics接口prometheus.yaml配置

- 获取全节点数据配置文件 prometheus.yml：

```
- job_name: 'rds_exporter'

metrics_path: /metrics

static_configs:
  - targets:
      - rds

relabel_configs:
  - source_labels: [__address__]
    target_label: __param_target
```

```
- source_labels: [__param_target]
  target_label: instance
- target_label: __address__
  replacement: 192.168.0.92:9091
```

● 获取具体节点数据配置 prometheus.yml:

```
- job_name: 'rds_exporter'
metrics_path: /metrics
static_configs:
  - targets:
      - rds://192.168.0.86:6323
      - rds://192.168.0.86:6380
relabel_configs:
  - source_labels: [__address__]
    target_label: __param_target
  - source_labels: [__param_target]
    target_label: instance
  - target_label: __address__
    replacement: 192.168.0.92:9091
```

3.4.2 metrics2cn接口prometheus.yml配置

获取全节点数据配置 prometheus.yml:

```
- job_name: 'rds_exporter'
metrics_path: /metrics2cn
static_configs:
  - targets:
      - 192.168.0.200:9091
metric_relabel_configs:
  - source_labels: [exported_instance]
```

target_label: instance

3.5 数据展示

- 直接在 prometheus 上展示

The screenshot shows the Prometheus web interface. The 'Targets' tab is selected, displaying a list of scrape targets. Two target groups are visible: 'EXPORTER01 (0/1 up)' and 'rds_exporter (3/3 up)'. The 'rds_exporter' group contains three targets, all in an 'UP' state. Each target has a unique label 'instance' representing its IP and port, and a common label 'job' set to 'rds_exporter'. The table also shows the last scrape time, duration, and any errors.

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.0.92:9091/metrics target='rds://192.168.0.86:6323'	UP	instance='rds://192.168.0.86:6323' job='rds_exporter'	4.22s ago	1.499ms	
http://192.168.0.92:9091/metrics target='rds://192.168.0.86:6380'	UP	instance='rds://192.168.0.86:6380' job='rds_exporter'	5.26s ago	1.695ms	
http://192.168.0.92:9091/metrics target='RDS'	UP	instance='RDS' job='rds_exporter'	9.281s ago	1.458ms	

```
# redis_memcache_connections metric
# redis_memcache_connections gauge
redis_memcache_connections{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_uptime_in_seconds metric
# redis_uptime_in_seconds gauge
redis_uptime_in_seconds{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 437455.0
# redis_up metric
# redis_up gauge
redis_up{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 1.0
# redis_connections_received_total metric
# redis_connections_received_total gauge
redis_connections_received_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 102878.0
# redis_net_input_bytes_total metric
# redis_net_input_bytes_total gauge
redis_net_input_bytes_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_net_output_bytes_total metric
# redis_net_output_bytes_total gauge
redis_net_output_bytes_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_memory_used_bytes metric
# redis_memory_used_bytes gauge
redis_memory_used_bytes{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 2.05897648E8
# redis_expired_keys_total metric
# redis_expired_keys_total gauge
redis_expired_keys_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_rds_connections metric
# redis_rds_connections gauge
redis_rds_connections{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_memory_used_rds_bytes metric
# redis_memory_used_rds_bytes gauge
redis_memory_used_rds_bytes{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 2.05897648E8
# redis_slow_logs metric
# redis_slow_logs gauge
redis_slow_logs{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_license_occupied metric
# redis_license_occupied gauge
redis_license_occupied{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_commands_processed_total metric
# redis_commands_processed_total gauge
redis_commands_processed_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 30.0
# redis_cpu_user_seconds_total metric
# redis_cpu_user_seconds_total gauge
redis_cpu_user_seconds_total{pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",} 0.0
# redis_memory_free_bytes metric
# redis_memory_free_bytes gauge
```

```
# redis_memcache_connections metric
# redis_memcache_connections gauge
redis_memcache_connections(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_uptime_in_seconds metric
# redis_uptime_in_seconds gauge
redis_uptime_in_seconds(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 437455.0
# redis_up metric
# redis_up gauge
redis_up(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 1.0
# redis_connections_received_total metric
# redis_connections_received_total gauge
redis_connections_received_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 102878.0
# redis_net_input_bytes_total metric
# redis_net_input_bytes_total gauge
redis_net_input_bytes_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_net_output_bytes_total metric
# redis_net_output_bytes_total gauge
redis_net_output_bytes_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_memory_used_bytes metric
# redis_memory_used_bytes gauge
redis_memory_used_bytes(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 2.05897648E8
# redis_expired_keys_total metric
# redis_expired_keys_total gauge
redis_expired_keys_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_rds_connections metric
# redis_rds_connections gauge
redis_rds_connections(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_memory_used_rss_bytes metric
# redis_memory_used_rss_bytes gauge
redis_memory_used_rss_bytes(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 2.05897648E8
# redis_slow_logs metric
# redis_slow_logs gauge
redis_slow_logs(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_license_occupied metric
# redis_license_occupied gauge
redis_license_occupied(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_commands_processed_total metric
# redis_commands_processed_total gauge
redis_commands_processed_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 30.0
# redis_cpu_user_seconds_total metric
# redis_cpu_user_seconds_total gauge
redis_cpu_user_seconds_total(pod="S13_test13-192.168.0.86:6323", instance="rds://192.168.0.86:6323", namespace="S13_test13",) 0.0
# redis_memory_free_bytes metric
# redis_memory_free_bytes gauge
```

● 在 grafana 上展示

由于数据获取有两种方式，所以 grafana 的模板设置有两种：

- 第一种：使用 rds 修改后的模板，展示全部节点的监控数据。

以获取 namespace 为例，需设置获取条件 instance=" rds "，对应 prometheus.yml 里配置的 targets。

```
{
  "allValue": null,
  "current": {},
  "datasource": "$DS_PROMETHEUS",
  "definition": "label_values({instance=\"rds\"}, namespace)",
  "hide": 0,
  "includeAll": false,
  "label": "Namespace",
  "multi": false,
  "name": "namespace",
  "options": [],
  "query": "label_values({instance=\"rds\"}, namespace)",
  "refresh": 2,
  "regex": "",
  "skipUrlSync": false,
  "sort": 0,
  "tagValuesQuery": "",
  "tags": [],
  "tagsQuery": "",
  "type": "query",
  "useTags": false
},
```

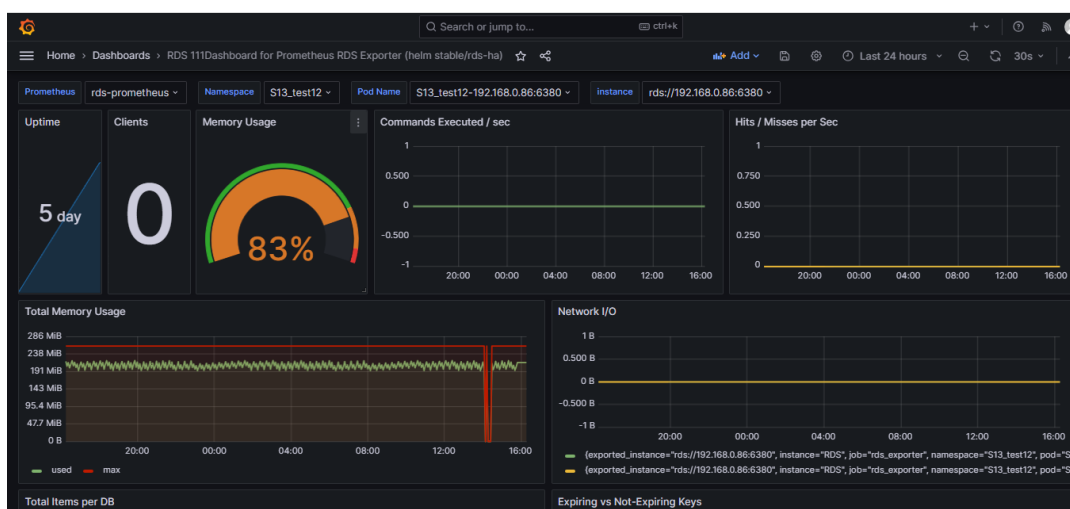
具体需要获取的指标配置需要增加动态后缀。


```

"options": {
  "dataLinks": [],
},
"percentage": false,
"pointradius": 5,
"points": false,
"renderer": "flot",
"seriesOverrides": [],
"spaceLength": 10,
"stack": true,
"steppedLine": false,
"targets": [
  {
    "expr": "sum (redis_db_keys{namespace=${exported_instance=\"${instance}\"}}",
    "format": "time_series",
    "interval": "",
    "intervalFactor": 2,
    "legendFormat": "{{ db }}",
    "refId": "A",
    "step": 240,
    "target": ""
  }
],
"thresholds": [],

```

设置模板成功之后，将 rds_grafana.json 导入到 grafana 的 Dashboards 中，获取数据之后展示为：

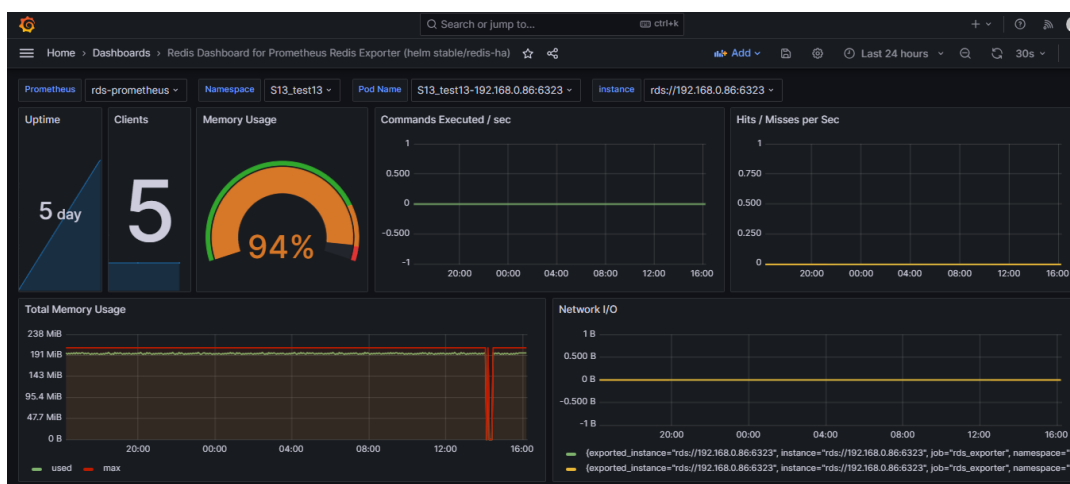


- 第二种方式是使用 redis 的模板，但是只能展示一个节点的数据。

Redis 模板需要修改 instance 的配置：

```
{
  "allValue": null,
  "current": {},
  "datasource": "${DS_PROMETHEUS}",
  "definition": "label_values(redis_up{namespace=\"${namespace}\", pod=\"${pod_name}\", exported_instance",
  "hide": 0,
  "includeAll": false,
  "label": null,
  "multi": false,
  "name": "instance",
  "options": [],
  "query": "label_values(redis_up{namespace=\"${namespace}\", pod=\"${pod_name}\", exported_instance",
  "refresh": 2,
  "regex": "",
  "skipUrlSync": false,
  "sort": 1,
  "tagValuesQuery": "",
  "tags": [],
  "tagsQuery": "",
  "type": "query",
  "useTags": false
}
```

设置成功后，将 json 导入 grafana 中，展示效果为：



附录A 术语说明

名称	说明
TongRDS (简称 RDS)	东方通分布式内存数据缓存中间件

